Análisis del problema

Situación y propósito

La plataforma necesita trabajar con proveedores que realizan tareas diferentes: cobrar compras, gestionar entregas y preparar documentos de facturación. El problema consiste en coordinarlos desde un mismo sistema y permitir que se agreguen nuevos proveedores sin cambiar el motor principal que procesa las solicitudes.

La solución utiliza un catálogo compartido donde cada integración tiene identificador, nombre, dirección del servicio, estado de conexión y latencia promedio. El administrador puede consultar ese catálogo, encontrar un proveedor por su identificador y ordenar los resultados para reconocer los servicios más lentos.

Cómo se resolverá

Al iniciar se registran diez integraciones: cuatro pasarelas de pago, tres proveedores logísticos y tres certificadores FEL. Todas se guardan en una sola lista. Aunque los objetos tienen funciones distintas, comparten una clase base y contratos que permiten tratarlos de una manera uniforme.

Las interfaces Transaccional, Documentable y Auditable establecen las operaciones comunes. Los pagos realizan cobros y reintegros; logística cotiza, rastrea y genera guías; FEL produce XML de ejemplo. Las operaciones que un proveedor no admite devuelven un mensaje de omisión. Así, pedir un reintegro a logística no detiene el programa.

Interacción con el administrador

Un menú de consola permite listar y buscar integraciones, ordenar por latencia, consultar estados, simular cobros y reintegros, generar documentos, revisar el historial, cambiar la conexión, cotizar envíos y rastrear paquetes. Los identificadores inexistentes y las entradas incorrectas se atienden con mensajes que permiten continuar.

Alcance del prototipo

Las APIs y sus respuestas se simulan en memoria. La latencia y la conexión son datos configurados, no mediciones de red. No se necesita acceso a internet, credenciales ni servicios contratados. Los documentos no tienen validez legal. El programa no conserva los datos cuando se cierra.

Clases y responsabilidades Atributos

IntegracionAPI (abstracta): Reúne identificador, nombre, endpoint, estado, latenciaPromedio e historial. Consulta configuración y estado, registra eventos, compara latencias y define identidad por identificador. Los datos fijos se leen mediante getters; estado y latencia tienen setters validados.

PasarelaPago (abstracta): Agrega saldoReintegrable para controlar cuánto dinero cobrado puede devolverse. Implementa cobrar y reintegrar con BigDecimal para evitar errores de representación decimal. Rechaza montos inválidos, desconexiones y reintegros mayores al saldo disponible.

PasarelaTarjeta, TransferenciaBancaria y BilleteraDigital: Son clases concretas de pago. Heredan la lógica financiera y reciben su configuración en el constructor. Su tipo identifica el canal utilizado; no necesitan duplicar atributos ni las reglas compartidas.

ProveedorLogistico: Agrega tarifaPorKg. Calcula cotizaciones, simula rastreos y genera guías de envío. Conserva las operaciones financieras neutrales del contrato Transaccional.

CertificadorFEL: Agrega nitEmisor. Genera un XML educativo asociado a una referencia de pedido y escapa caracteres especiales. Mantiene las operaciones financieras neutrales.

CatalogoIntegraciones: Tiene integraciones: ArrayList, la única lista del sistema. Agrega objetos sin repetir identificadores, busca, lista y ordena. Sus dos versiones de listar permiten mostrar todo o únicamente lo conectado sin construir otra lista.

MotorCheckout y CargaInicial: MotorCheckout recibe interfaces como parámetros y delega cobros, reintegros, documentos y consultas del historial. CargaInicial instancia los diez proveedores ficticios y los agrega al catálogo.

VistaConsola, ControladorIntegraciones y Main: La vista posee entrada: Scanner y se encarga del menú y la lectura validada. El controlador conserva referencias a catálogo, vista y motor para coordinar cada acción. Main crea esos objetos, realiza la carga inicial e inicia el controlador.

Decisiones de diseño

Contratos con comportamiento neutral

Transaccional incluye cobrar y reintegrar como métodos default que devuelven IGNORADO. Documentable hace lo mismo para generarDocumento. IntegracionAPI adopta estos contratos y las clases que sí ofrecen una función la sobrescriben. Esta adaptación uniforme responde al requisito de aceptar una petición no compatible sin lanzar una excepción. Auditable exige registrarEvento y consultarHistorial.

Una única lista polimórfica

guarda objetos de distintas subclases en ArrayList. No mantiene colecciones separadas por servicio y no devuelve copias de la lista. Los filtros producen texto. El historial utiliza StringBuilder, no otra lista.

Polimorfismo y sobrecarga

MotorCheckout.procesar recibe Transaccional; documentar recibe Documentable y auditar recibe Auditable. El objeto concreto decide la implementación. procesar(servicio, monto) realiza un cobro; procesar(servicio, monto, esReintegro) permite seleccionar cobro o reintegro. listar() y listar(soloConectadas) son otra sobrecarga útil.

Herencia, identidad y ordenamiento

PasarelaPago reutiliza la configuración de IntegracionAPI y comparte las reglas financieras con sus tres subclases. Comparable compara latencias en orden ascendente y usa el ID para desempatar; Collections.sort ordena la misma lista. equals y hashCode usan el identificador inmutable. El catálogo impide que coexistan identificadores repetidos.

Separación de responsabilidades

El modelo reúne integraciones, catálogo, reglas y motor. VistaConsola presenta mensajes y lee datos, sin decidir reglas financieras. ControladorIntegraciones busca la integración y solicita al modelo la operación elegida. Solo las funciones específicas de cotizar y rastrear comprueban el tipo logístico; el motor no reconoce clases concretas de proveedores.

Extensión y límites

Una nueva startup con las mismas funciones se registra como otro ProveedorLogistico. Si necesita comportamiento propio, se crea una subclase y se

sobrescriben sus operaciones. El motor de checkout permanece igual. El reintegro se limita al saldo acumulado de cada pasarela; este prototipo no lleva conciliación por pedido ni transacciones reales.

UML

